



forward together
sonke siya phambili
saam vorentoe

DESIGN (E) 314 TECHNICAL REPORT

Smart Prepaid Energy Controller

Author:
CARL VAN ROOYEN

Student Number:
27005348

May 19, 2025

Plagiaatverklaring / Plagiarism Declaration

1. Plagiaat is die oorneem en gebruik van die idees, materiaal en ander intellektuele eiendom van ander persone asof dit jou eie werk is.

Plagiarism is the use of ideas, material and other intellectual property of another's work and to present is as my own.

2. Ek erken dat die pleeg van plagiaat 'n strafbare oortreding is aangesien dit 'n vorm van diefstal is.

I agree that plagiarism is a punishable offence because it constitutes theft.

3. Ek verstaan ook dat direkte vertalings plagiaat is.

I also understand that direct translations are plagiarism.

4. Dienooreenkomstig is alle aanhalings en bydraes vanuit enige bron (ingesluit die internet) volledig verwys (erken). Ek erken dat die woordelike aanhaal van teks sonder aanhalingstekens (selfs al word die bron volledig erken) plagiaat is.

Accordingly all quotations and contributions from any source whatsoever (including the internet) have been cited fully. I understand that the reproduction of text without quotation marks (even when the source is cited) is plagiarism.

5. Ek verklaar dat die werk in hierdie skryfstuk vervat, behalwe waar anders aangedui, my eie oorspronklike werk is en dat ek dit nie vantevore in die geheel of gedeeltelik ingehandig het vir bepunting in hierdie module/werkstuk of 'n ander module/werkstuk nie.

I declare that the work contained in this assignment, except where otherwise stated, is my original work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.

Handtekening / *Signature*

Studentenommer / *Student number*

Voorletters en van / *Initials and surname*

Datum / *Date*

Abstract

This report details the design and creation of a smart prepaid power meter. This system uses an embedded system to act as an electricity meter in rentail properties. The device is able to measure the power consumed by the tenant, and use this information to provide energy metering. Though the intention behind this device is to track energy usage, it also performs many other functionalities, such as load control, live power measurement reporting, user authentication via RFID, and more.

This report details the design choices made to develop the hardware and software of the system. Additionally, the success of this system against user requirements through the use of measurements. Ultimately, the energy controller is successful in energy metering, but this report contains reccomendations towards further improvements towards the system.

Contents

1	Introduction	1
2	System description	2
3	Hardware design and implementation	2
3.1	Hardware Block Diagram and Description of Interaction	2
3.2	Power Supply	3
3.3	LEDs (Debug)	4
3.4	Signal Conditioning and Measurement	4
3.5	Load Control Circuit	6
3.6	Keypad and button	7
3.7	OLED	7
3.8	SD Card	7
3.9	RFID	8
4	Software design and implementation	8
4.1	Software Block diagram and description of interaction	8
4.2	ADC setup and sampling data management	10
4.3	Analogue measurements, voltage, current, and phase	11
4.4	Power calculations, P, S, Q, and PF	12
4.5	UART communications	13
4.6	Keypad driver setup and button bounce handling	14
4.7	OLED low-level interface	14
4.8	FSM control and logic	15
4.9	Energy consumption calculations and RTC	16
4.10	SD Card Interface and Logging	16
4.11	RFID Interface and authentication	17
5	Measurements and Results	18
5.1	Power Supply	18
5.2	V and I circuits	19
5.3	V, I, Phase, P, S, Q	20
5.4	Load Circuit	21
5.5	Keypad and Button	21
5.6	UART Communications	22
5.7	OLED and Menu Functionality	23
5.8	SD Card	23
5.9	RFID	24
5.10	Full System	24
6	Conclusions	24
6.1	System Compliance/Non-Compliance	24
6.2	Improvements to be Made	25
6.3	Final Thoughts	25

List of Figures

1	System Hardware Regions	2
2	Power Supply Circuit	3
3	General Voltage Regulator [1]	3
4	Debug Setup	4
5	General Design for Conditioning Circuit [2]	4

6	Monte-Carlo Simulation of Current Circuit, 5% Tolerance Resistors	5
7	Relay and Load	6
8	Keypad Pinout [3]	7
9	Emergency Button Circuit	7
10	OLED Screen	7
11	SPI Micro SD Card Reader	7
12	RC522 RFID Reader	8
13	High-Level View of Software Flow	9
14	Program Flow to Control Voltage and Current Measurements	11
15	Steps to process data	11
16	UART Flow Diagram	13
17	Keypad Debouncing	14
18	Keypad and Button Software Handling	14
19	Finite State Machine States and Transitions	15
20	Main Loop and Interrupt Control	15
21	SD Card Flow Diagram	17
22	RFID Authentication Process	17
23	Setup for measuring 3.3V Regulator	18
24	3.3V Maximally Loaded	18
25	3.3V Maximally Loaded (Zoomed)	18
26	Op-Amp Output at Max Inputs	19
27	Op-Amp Output Zoomed (V=yellow, I=blue)	19
28	Testing Op-Amp Linearity	19
29	Op-Amp Output Zoomed (V=yellow, I=blue)	19
30	Testing Setup for Signal Generator 1	20
31	Testing Setup for Signal Generator 2	20
32	Voltage and Current Sweep Over Full Range	21
33	Phase Sweep Over Full Range	21
34	Minimum Mosfet Input Test	21
35	Verifying Flyback Diode behavior	21
36	Testing Inrush Resistor	21
37	Keypad Bounce Test	22
38	Button Bounce Test	22
39	UART Transmit	22
40	Two Commands, No break in Receiving	22
41	Command In Middle of UART Message	22
42	Really Long Message with Command	22
43	All OLED Screens	23
44	SD Card Access (Logging)	23
45	Timing Logging Rate	23
46	RFID Authentication Screen	24
47	RFID accessing Tag's UID	24

List of Tables

1	User Requirements	1
2	Resistor Values per Circuit	4
3	Valid UART Commands and Responses	13
4	V, I, and Phase Results	20

List of Abbreviations

ADC analog to digital converter

GPIO general purpuse input-output

IC integrated circuit

LED light emitting diode

NMOS n-channel metal oxide semiconductor

1 Introduction

The students of E-Design 314 2025 have been tasked with designing and building a smart prepaid power meter to be used by landlords with tenants. It aims to provide both more control to the landlord by allowing them to keep track of the tenant's power usage, while also giving peace of mind to the tenant, by ensuring that they never get overcharged for power they don't use. The system can track power usage through a single 3-prong wall socket, as well as control the flow of electricity through said socket. It will be able to load units, just like an Eskom-provided power meter, allowing the tenant to buy the exact amount of power from their landlord that they need. Once credits run out, the prepaid meter can turn off the power through the wall socket.

From this brief, a number of user requirements have been given for the student's system's to follow. These are expanded upon, and detailed in the following section.

	Requirement	Description
UR1	Internal Power Supply	System must generate regulated supplies from a nominal 9V–12V battery or DC power supply.
UR2	UART Communication	Must support UART communication (transmit and receive) using 8E1 format at 57600 bit/s.
UR3	Power Measurements	Must measure: • Voltage to 5% accuracy • Current to 5% accuracy • Phase to 5% accuracy to then calculate complex power.
UR4	Energy Usage	Must determine energy consumed over time.
UR5	Load Switching	Must control a load switch (relay) to toggle the electrical load, using software control and a physical emergency button override.
UR6	Debug LEDs	Must use status-indicating LEDs as output.
UR7	Keypad	Must use a keypad for user input on system settings and display modes.
UR8	OLED Display	Must use an OLED display as the primary visual output.
UR9	SD Card	Must log measured and calculated data to an SD card
UR10	RFID	Must implement user authentication using an RFID system
UR11	Real Time Clock	Must keep accurate time using an RTC (internal to the microprocessor).

Table 1: User Requirements

2 System description

3 Hardware design and implementation

3.1 Hardware Block Diagram and Description of Interaction

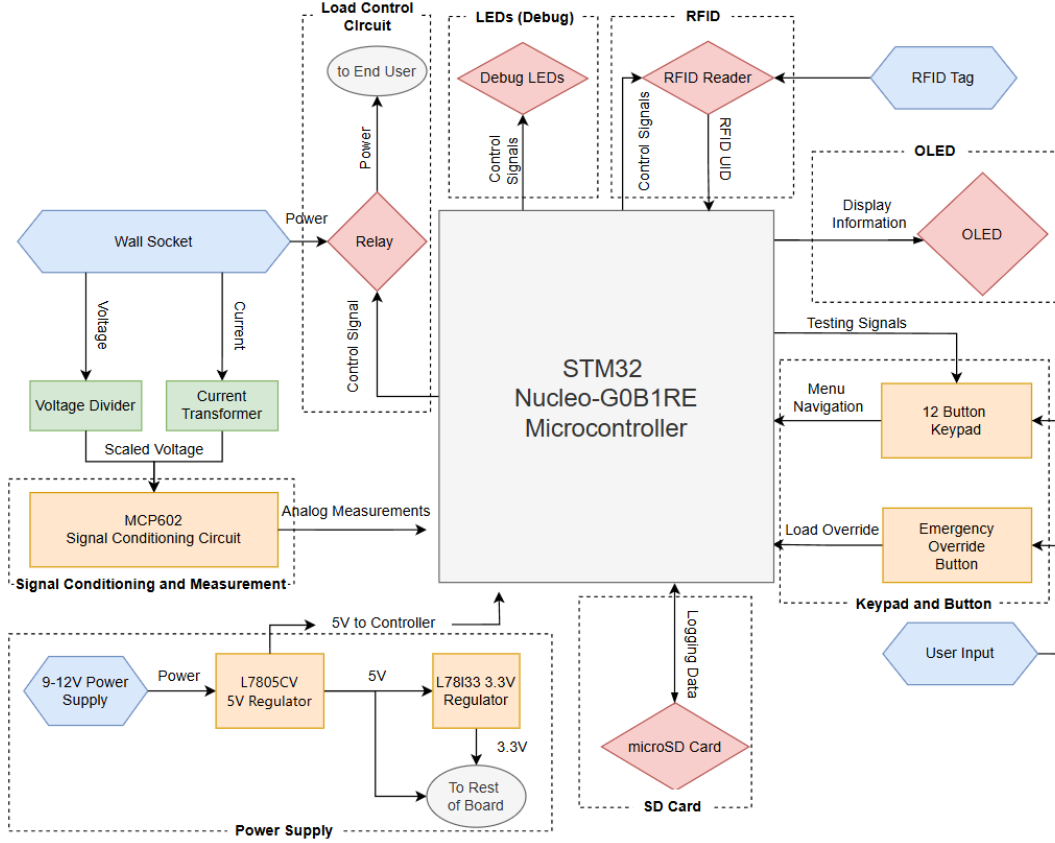


Figure 1: System Hardware Regions

The smart power meter, at its core, is controlled by a STM32G0B1RE microcontroller. The system is powered through a 2155 female DC jack, and is designed to take between 9-12V as input, allowing it to be powered by a battery or dedicated DC power supply. This power is stepped down to 5V and 3.3V to be used within the power meter - for the microcontroller and peripherals.

The system is broken up into eight sub-circuits, shown in figure 1. The first of these is the power supply. The L7805CV regulator takes the input DC voltage, and steps it down to a 5V rail voltage to be used throughout the system [4]. This 5V supply is then fed into the L78133 regulator, which steps it down to a further 3.3V [5] for lower voltage peripherals across the device.

The second is measurement and signal conditioning: Two external analog voltage signals (directly proportional to the voltage and current of the wall socket, respectively) come into the conditioning circuit, which are then biased and scaled so that the onboard analog to digital converter (ADC) on the microcontroller can read the signals. This biasing and scaling is performed by an MCP602 - an integrated circuit (IC) with two op-amps built in [6]. The input signals can then be used to calculate power used by the tenant.

The power flow circuit consists of a J107 relay, controlled by a 2N7000 n-channel metal oxide semiconductor (NMOS). The relay acts as a high-voltage switch, allowing the wall socket to be connected and disconnected, so that the prepaid meter can control the power flow. The 2N7000 is used to turn the relay on and off, as the microcontroller cannot provide enough current to do so itself.

The LED debug circuitry is a set of 4 light emitting diodes (LEDs) that are used to provide visual feedback to the user on what the system is currently doing. They are controlled and powered directly from the microcontroller.

The OLED is a screen, powered by the SSD1306 controller, that is used to provide visual feedback to the end user about what is going on with the system - measurements that the system is taking, units left, etc.

The SD card is used as permanent storage for the system, for logging purposes as well as a means to maintain state between resets. The physical hardware consists of a simple adaptor, that allows the use of SPI to communicate with the SD card.

The RFID reader gives the system the ability to grant administrative access to sensitive commands, like the ability to add units and the ability to turn off unit counting. The hardware required for this is the RC552 chip - which communicates over SPI by default.

Lastly, the keypad and button are used as physical input to the system - The keypad for navigating menus, and the button for emergency control over the load. There is no fancy hardware required to make these two components work (apart from the keypad and button themselves), and all the heavy lifting is dealt with in software.

3.2 Power Supply

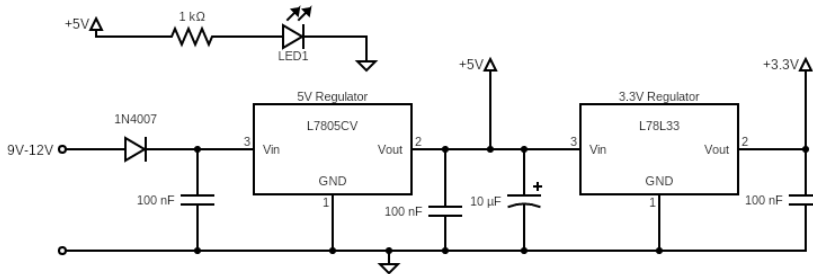
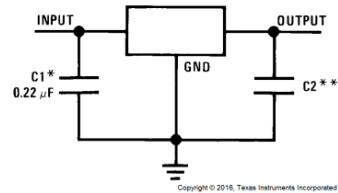


Figure 2: Power Supply Circuit

Fixed Output Voltage Regulator



*Required if the regulator is located far from the power supply filter.
 **Although no output capacitor is needed for stability, it does help transient response. (If needed, use 0.1-µF, ceramic disc).

Figure 3: General Voltage Regulator [1]

There are two factors that lead to the need for an onboard regulated power supply: Firstly, we need both 5V and 3.3V to be able to power the different components and peripherals on the board. Secondly, we need to be able to take in a varying voltage range [UR3](#), and be able to use batteries to power the device, which cannot be used directly as they can lose voltage over time.

The IC of choice here is a voltage regulator - they take a range of voltages in, and hold a constant voltage at the output. The dropout voltage of the L7805CV is $2.5V_{\max}$ [\[4\]](#), which makes it suitable for the 9V minimum input voltage, as it leaves a leeway of 1.5V. This regulator is rated for 35V maximum [\[4\]](#), and since it is the only component on the board that receives the DC input, this 35V is our system's theoretical maximum input. The L78L33 regulator has a dropout voltage of 1.7V [\[5\]](#), which is just barely enough to get a 3.3V output from a 5V input.

The diode on the input acts as protection if the end user ever plugs the power supply in backwards. The exact choice of resistors C1 and C2 (Figure 3) are arbitrary, as long as the input is small enough to block high-frequency noise into the regulator, and the output is small enough to do the same for transient outputs. The 10µF capacitor on the 5V output is added to provide current to both the rest of the 5V rail and the 3.3V regulator during current surges.

LED1 is used as an indication that the 5V regulator is working (the board is receiving power). A current limiting resistor is placed in series with it, limiting the current flow to 3mA.

3.3 LEDs (Debug)

User requirement 6 dictates that the system must use a set of debug LEDs as one of the outputs of our system. LEDs are useful, as they provide meaningful information at a glance, without needing to navigate any menus. Our system employs four controllable debug LEDs - The general design of which is shown in Figure 4.

The design of these LEDs is quite simple, as they draw low enough current to be powered straight from the general purpose input-output (GPIO) ports. They are rated for 20mA maximum[7], but can still produce acceptable brightness using far lower current. The GPIO pins can only source 15mA[8], so we could never reach this maximum current anyway. To determine the value for the resistor, we started at the lowest possible resistance ($R = \frac{3.3-2V}{15mA} \approx 87\Omega$), then increased the resistance until the brightness dipped below what was personally deemed acceptable. This is what led to the choice of 390 Ω (a current of 3.3mA).

The appendix shows which LEDs are connected to which GPIO pins, in the pinout.

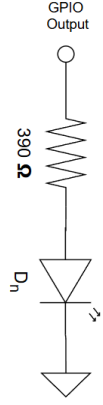


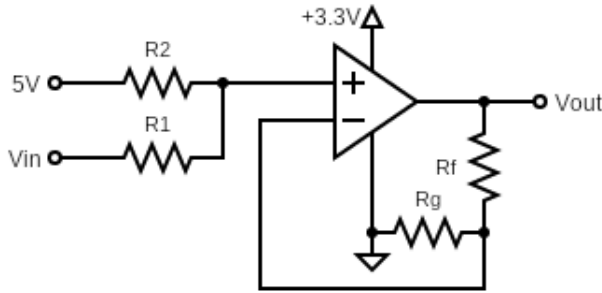
Figure 4:
Debug
Setup

3.4 Signal Conditioning and Measurement

The circuits used to measure the voltage and current from the wall both produce analog voltage signals. The voltage divider scales the wall voltage (230V_{rms}) to produce a 2V_{PP} signal, while the current transformer is tuned to produce a 2V_{PP} voltage when the current is 15A [9].

The microcontroller's onboard ADC can be used to measure these analog signals, but firstly the signals *must* be conditioned. The ADC can only read signals above 0V, so the signals must be biased positively to never be negative. Additionally, the ADC has limited resolution - by default, it is set to its maximum resolution of 12 bits, which, given its standard maximum limit of 3.3V [8], gives a resolution of $\approx 806\mu V/bit$ (our smallest input step size).

These limitations mean that we must move and scale our signal to fill as much of the 0V-3.3V range as we can reliably achieve, so that A: we do not damage the ADC with a negative voltage input and B: we can read our input signal as accurately as possible.



To Measure:	Voltage	Current
R_1	6.8k Ω	2.7k Ω
R_2	27k Ω	12k Ω
R_f	12k Ω	1.2k Ω
R_g	18k Ω	1.5k Ω
Gain	1.331V	1.677V
Offset	1.469V	1.653V

Table 2: Resistor Values per Circuit

Figure 5: General Design for Conditioning Circuit [2]

This can be achieved with a summing, non-inverting op-amp - the layout of which is shown in Figure 5. The equation for output voltage of this circuit is shown below.

$$V_{out} = \left(\frac{r_2}{r_1 + r_2} V_{in} + \frac{r_1}{r_1 + r_2} \cdot 5 \right) \left(1 + \frac{R_f}{R_g} \right)$$

A non-inverting op-amp design was used, as guided by [2]. I used the 5V rail as the vref voltage, as the 3.3V supply has a very small current limit of 100mA [5] and should therefore be used as sparingly as possible.

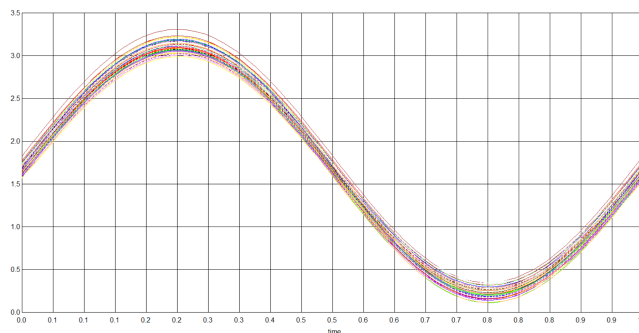


Figure 6: Monte-Carlo Simulation of Current Circuit, 5% Tolerance Resistors

The op-amp's power supply rails are connected to 3.3V and 0V respectively. This ensures that the op-amp cannot provide voltages outside the ADC's limits, protecting it in the case of extreme inputs. An MCP602 was used when building the physical circuit - it is an IC with 2 op-amps built in, so both circuits can share the same IC and power rails [6].

The gain and offset for each circuit was worked out using expected maximum and minimum inputs and outputs to the system. I gave both circuits an output range from 0.2V-3.1V, to leave tolerance for inexact resistor values. This effect is demonstrated in Figure 6, which shows a Monte-Carlo simulation of the current measuring circuit, varying the resistor values within $\pm 5\%$. To design the limits, I reran this simulation with differing output ranges until the output range was no longer outside of the target 0-3.3V.

The voltage measurement circuit takes $2V_{PP}$ in when the wall is at $230V_{RMS}$, except Eskom allows a $\pm 10\%$ tolerance around the typical 230V [10] - meaning the input voltage maximum is actually $2.2V_{PP}$. For the current measurements, the fuses in a home's wiring limit maximum current to 15A, so the signal is truly capped to $2V_{PP}$. Using these limits, we get an offset of 1.65V for both conditioning circuits ($\frac{3.1-0.2}{2}$). The voltage circuit has a gain of 1.32 and the current circuit has a gain of 1.45.

Plugging these values into the [Desmos script](#) I wrote, I obtained two sets of valid E12 resistor values to work with. These are tabulated in Table 2, along with the updated gain and offsets after changing the resistor values.

From Equation 3.4, it can be seen that the exact resistor values are not what determine the macroscopic behavior of this circuit - it is their ratio. This is important, as it means that R_1 and R_2 can be scaled together, as well as R_f and R_g , without effecting the behavior of the circuit.

The choice of resistor size is dictated by a few factors: Firstly, for R_1 and R_2 , if our resistors are too small, we risk pulling too large a current from both our measurement circuitry and our 5V rail. If we use resistors that are too large, we run into the physical limits of the MCP602: not enough bias current to turn on the op-amp, and a sluggish response due to internal capacitance being the main bottlenecks.

Similar limits apply to R_f and R_g : too small, and we are forcing the op-amp to deliver too much current. Too large and we are limiting how fast the ADC can be charged up by the conditioning circuitry, limiting sample times.

I made the mistake of not doing enough research before choosing my resistor values, and chose conservatively small values in the 1k-10k range. The power efficiency of my circuit would be greatly improved with larger resistors, for little downside.

3.5 Load Control Circuit

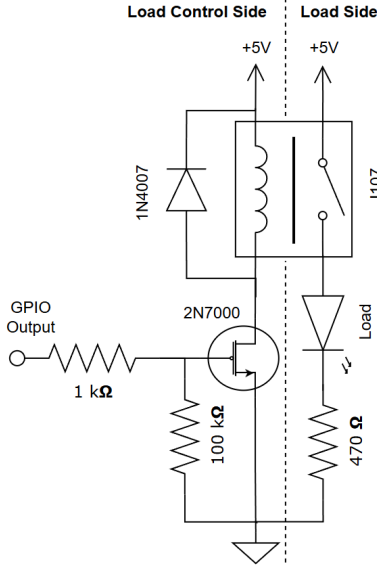


Figure 7: Relay and Load

In order to control the client's electrical load [UR5](#), the microcontroller needs to be able to turn on and off 230V at the wall. This is not possible with just the microcontroller, but a relay can be used as an electrically isolated way to control higher voltage loads.

The relay of choice is a normally open J107 - it is not rated for full AC mains [\[11\]](#), and uses 0.36W when active. Even though This specific relay isn't able to switch mains power, in a real circuit this could be solved by chaining this relay into an even larger one. The power used also poses an issue - at a perfect rail-rail input of 5V, we would be drawing minimum 72mA to keep the relay active. This is far too much for the microcontroller[\[12\]](#), so we need to source that current elsewhere.

An easy solution is to use a MOSFET - by applying a control voltage at the gate, we can drive a large current at the source/drain, while also drawing no current from the GPIO pin. A spice simulation shows that the 2N7000 NMOS transistor pushes $\approx 47\text{mA}$ with an input of 3.3V. This may seem low, but testing of my relay showed that it would work with currents as low as ..

The next design consideration we need to make is that the NMOS's gate capacitance draws a spike of current while it charges up. The peak of this current, known as the inrush current, needs to be limited. The peak inrush current occurs when the gate capacitor is completely discharged (at 0V), and is worked out as:

$$i_{\max} = \frac{3.3 - 0\text{ V}}{R_{\text{inrush}}} \Rightarrow i_{\max} = 3.3\text{ mA}, R_{\text{inrush}} = 1\text{ k}\Omega$$

We also do not want to turn on our output while our system is off/in startup, so we add a hardware pull-down resistor to pull the gate of the transistor to ground when it isn't powered.

Our inrush and pull-down resistor form a voltage divider when the gate isn't drawing current, so in order to keep our gate voltage as high as possible, I chose a pull-down that was 100x larger than the inrush resistor. This gives a theoretical input $V_{gs} = 3.3\text{ V} \cdot \frac{100\text{ k}\Omega}{100\text{ k}\Omega + 1\text{ k}\Omega} = 3.27\text{ V}$

The 1N4007 is a fly-back diode that provides a return path for the relay current, once the transistor has been deactivated. The internal solenoid of the relay is basically a large inductor, and so sudden drops in current will lead to large voltage spikes - the diode provides a path to mitigate the damage these spikes have on the rest of our system.

Lastly, an LED has been put on the load side of the circuit, simulating an electrical load. The 470Ω resistor again limits the current through this LED - this time to $\frac{5-2\text{ V}}{470\Omega} = 6.38\text{ mA}$. This is higher than the other LEDs in the system as I used a green LED here.

3.6 Keypad and button

SPARKFUN ELECTRONICS
12-BUTTON KEYPAD COM-14662

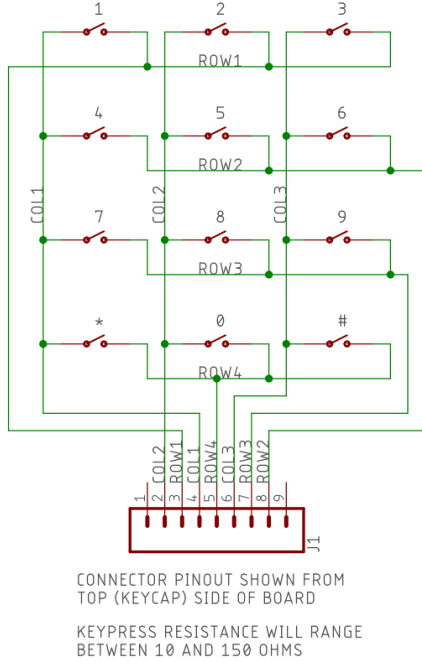


Figure 8: Keypad Pinout [3]

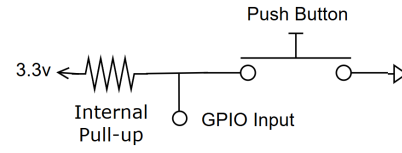


Figure 9: Emergency Button Circuit

The system requires that user input be given (UR5 and UR7). This is done in the form of a keypad (for navigating the OLED screens), and a button for emergency load control, which instantly toggles the load state.

The keypad is effectively 12 buttons that all should be debounced. The keypad pins are laid out as an array, with 7 pins corresponding to each row and column. To read from the keypad, you must first send an output on either a row or column, then read the corresponding column/row to figure out if a button is pressed or not. I chose to send signals on the rows (output), and read in on the columns (input), using an active high configuration. This means that if I read a high input (3.3V), the key is being pressed.

In order to simplify the hardware, all debouncing is handled in software, so both the keypad and button are wired directly to GPIO inputs/outputs. The button is set to be active low, so I have used an internal pull-up resistor on the GPIO input in order for it to be high when the button isn't pressed. The keypad is active high, so my inputs (columns) all have internal pull-downs resistors, for the same rationale.

3.7 OLED



Figure 10: OLED Screen

User requirement 8 stated that the system must use an OLED display as the primary visual output. We used a 128x32 OLED for this task, which is a black-and-white display, powered by the SSD1306 controller. The screen communicates over I2C, and requires no external passive components in order to function. I chose to power it over 3.3V, but it can also be powered at 5V.

The SSD1306 controller's I2C implementation handles both standard (100 kHz) and fast (400 kHz) modes [13]. I run the screen in fast mode, in order to limit the time taken sending data to the screen.

3.8 SD Card

In order to satisfy UR9, our system makes use of an SPI enclosure for a micro SD card. The driving reason behind the need for permanent storage is that, especially within the context of load shedding, our system must be able to maintain information between resets - most importantly, the amount of units left. The SD card is also useful as a debugging tool for any theoretical technicians that may be working with our system - it enables us to log what the system is receiving each second.

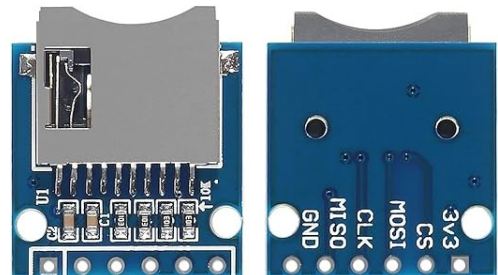


Figure 11: SPI Micro SD Card Reader

The SD card specification allows communication to the card over an SPI interface. The SD card's CLK line starts off at a slower clock speed (about 250Kbit/s in my code). Then, once the SD card has been initialized, the CLK line switches to its normal operating speed (which I have set to be 8MBits/s).

The SD and RFID reader both communicate over SPI, but due to the differing (and varying) clock speeds between the two components, it is simpler to keep their SPI lines separate, even if the SPI protocol allows for multiple devices connected to the same bus.

The only external component needed for this SD card is a large capacitor, between VCC and GND. This is because the SD card draws a lot of current when reading and writing (Demonstrated in Section 5.9), and the capacitor limits how much this affects our 3.3V regulator. I used a 10 μ F capacitor.

3.9 RFID

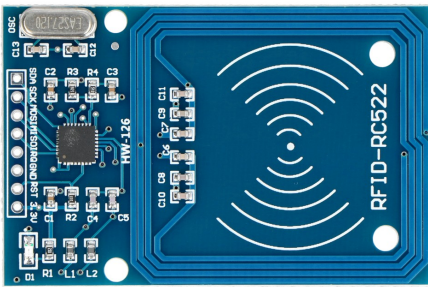


Figure 12: RC522 RFID Reader

A main component of any prepaid meter is the ability to add units. For our system, the end user buys power from their landlord, and then the landlord can come and enter new units for the user. To verify this process, and to prevent the end user from just doing this themselves, UR10 requires the system to use RFID authentication. The component of choice here is an RC522 RFID reader. It can communicate over a range of communication protocols, but the one we use is SPI, the default mode.

The reader is just plug-and-play: The only quirks are the SDA, IRQ, and RST pins. SDA is used as chip select when in SPI mode, and can be plugged into any GPIO output. IRQ is the reader's interrupt output, and is deactivated by default. I left the pin floating. The RST pin is used to reset the reader after a power-down, and can also be connected to any GPIO output.

The RFID reader creates a lot of noise in our system when the antenna is active, as the electromagnetic waves interact with wires and traces in our system, inducing emf and creating noise. I solved this by using a piece of aluminium foil, wrapped in cling film (to prevent any shorts from occurring), and built an enclosure around the SD card reader. This acts as a makeshift faraday cage, blocking and absorbing the electromagnetic waves produced by the antenna before they can interfere with the rest of the system. Proof of this working is shown in section 5.1.

4 Software design and implementation

4.1 Software Block diagram and description of interaction

The main purpose of our system is to act as a smart power meter. This means that it should be able to measure power, track energy usage over time, relay this information back to the user, allow the user to add units, and be able to control the connected load. Every hardware and software component of the system exists to serve these goals.

There are 7 main peripherals that the system has to communicate with: the UART, keypad, ADC, emergency button, RTC, RFID reader and SD card. These, in turn need to be handled by the software to produce output that satisfies the user requirements. The software features have been categorized into 10 main sections, and the rest of section 4 covers each more in-depth. Figure 13 shows all inputs to the software, outputs, and a general idea of how each subroutine interacts with each other. In order, here is a general explanation on what each subroutine does:

The finite state machine and control section is the main bulk of the program - it handles all the internal logic and state tracking required to have usable user input and output and persistent information.

The keypad driver converts keypad presses into keys that the finite state machine can process, in order to advance state (change menus, enter units, etc.).

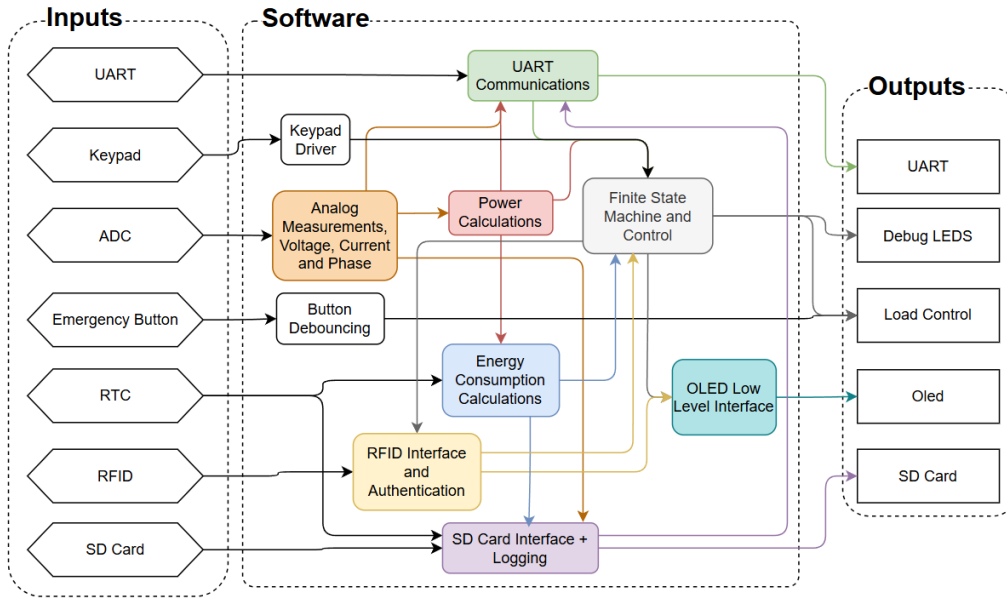


Figure 13: High-Level View of Software Flow

The button debouncing is done so that each press of the emergency button is only handled once - this is crucial during an emergency, i.e. an electrical fire.

The analog measurement portion of the code is used to get the software to read what real-world currents and voltages the system is plugged into, which is a step towards working out the power being drawn by the user at any moment in time. Again, this is crucial to a power meter, whose job is tracking power usage.

Power calculations are done using the analog measurements in order to find the power at any moment in time - important for the same reasoning as before.

Once power is known, the energy consumption calculation comes in. Combined with power information and an accurate count of time, the system is able to work out how much energy the end user has spent over time, and further use this information to keep track of the amount of units the user has left.

The SD card interface and logging are of vital importance for the system in the case of power loss. If the power ever cuts, the meter must be able to remember its state before power was lost. The SD card and logging system solve this by keeping track of how many units are left each second. This also doubles as a useful logging system for debugging or interests sake. Say the end user was using solar - keeping a log allows the user to analyze their power usage and shift their loads around according to daily demand, saving money in the long run. The interface layer is necessary in order for the microcontroller to communicate with the micro SD card.

The OLED interface is needed since the system requires a visual output. Without this subsystem, the meter cannot display any information back to the user on-device.

RFID is used to give administrative access to the system - this is great for landlords, as it allows them full control of their tenant's system. This is also a cheap way to implement adding units, instead of having to verify with some external server, like how normal meters work.

Lastly, a UART interface is used for debugging purposes. While this is not strictly necessary for the end user, it is helpful for, say, technicians who would have to work with this power meter in the real world. Adding this functionality allows easy access for a separate computer to talk to our power meter.

To simplify the explanation of my software, and in the interest of time, I have elected to omit a large majority of my error checking/handling from this report. Certain subsystems, like the SD card, have a large amount of error checking built in to prevent system from crashing (like in the case of removing the SD card mid-operation).

4.2 ADC setup and sampling data management

In order to measure the voltage and current from our signal conditioning circuit, there are a few requirements that our code must meet:

1. From the user requirements, the measured voltage and current must be within 5% of their real values - therefore we should introduce as little error as possible in software, especially since the hardware is already noisy.
2. The system has limited memory available to us. It can't store infinite samples for more accuracy.
3. We have limited CPU time available to us, so we need to limit the size of the buffer storing the samples: The more samples we have, the longer it will take us to process all of the sampled data.

Our main design parameter that we can control in software during setup is the number of samples we take per period. We want to minimize this as much as possible, to still get acceptable tolerance. The loss of accuracy from sampling rate would be due to a sample missing the exact peak or trough of our signal. In the absolute worst case, we miss this peak/trough by exactly half the period of our sample rate, as both the sample before the peak and after it would be equally far from the true value. Doing another [Desmos Simulation](#) shows this effect in action.

From the same simulation, we can work out this worst case error: At 100 samples, with the wave centered at $t=0s$, the worst case error is $\approx 0.05\%$.

$$\%Error = \left| \frac{\text{Measured val} - \text{True value}}{\text{True value}} \right| \cdot 100\% = \left| \frac{\cos\left(\frac{1}{2} \cdot \frac{2\pi}{n_{\text{samples}}}\right) - \cos(0)}{\cos(0)} \right| \cdot 100\%$$

This was deemed acceptable, as it should have almost no effect on our overall accuracy. At 100 samples per period of our 50Hz input wave, our sample rate must be 5kHz.

I chose to use a timer to trigger the ADC, as continuous conversion proved to be far too fast to be slowed to 5kHz. To get the timer to trigger at 5kHz, values for the pre-scaler and auto-reload registers had to be calculated.

The input clock f_{clk} was 64MHz. Choosing an arbitrary PSC value of 15 gives an ARR value of 799:

$$f_{out} = \frac{f_{clk}}{(PSC + 1)(ARR + 1)}$$

The ADC was initialized in scan conversion mode, with DMA continuous requests active. Each sample is set to take 12.5 cycles (to allow for more accurate sampling), and the ADC samples twice every time the timer trigger event occurs.

The DMA is set to circular mode, peripheral to memory. It increments the memory address after each transfer, and each transfer is half-word to half-word (since the ADC is 12 bits).

This setup means that the ADC can continuously sample at 5kHz (at 2 samples per clock tick), while the DMA sends the data to a circular buffer, constantly overwriting old measurements.

In order to have the half and full interrupts trigger after every period of the signal, the buffer must be 400 half-words long. 200 for the first half (of which 100 samples are voltage, and the other 100 are the current)

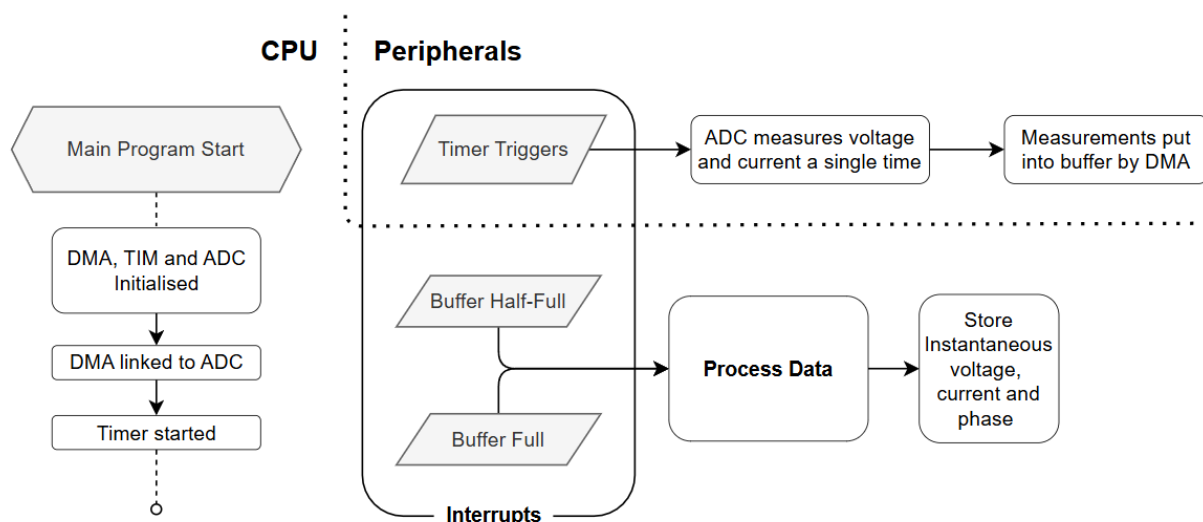


Figure 14: Program Flow to Control Voltage and Current Measurements

Figure 14 demonstrates the parts of the main program that take care of measuring the voltage and current.

4.3 Analogue measurements, voltage, current, and phase

Using the DMA in circular mode allows us to make use of buffer half-full and full interrupts. Whenever one half of the buffer fills up, the program stops normal flow and processes half of the values sitting in the buffer, while the ADC continues to fill up the other half. This allows us to do calculations on our buffer without risking some of the data being overwritten mid-operation, and without needing to pause the ADC.

Figure 15 shows the steps necessary to process the measured data. In order to read the voltage, current, and phase difference of the two waves, the following procedure is used:

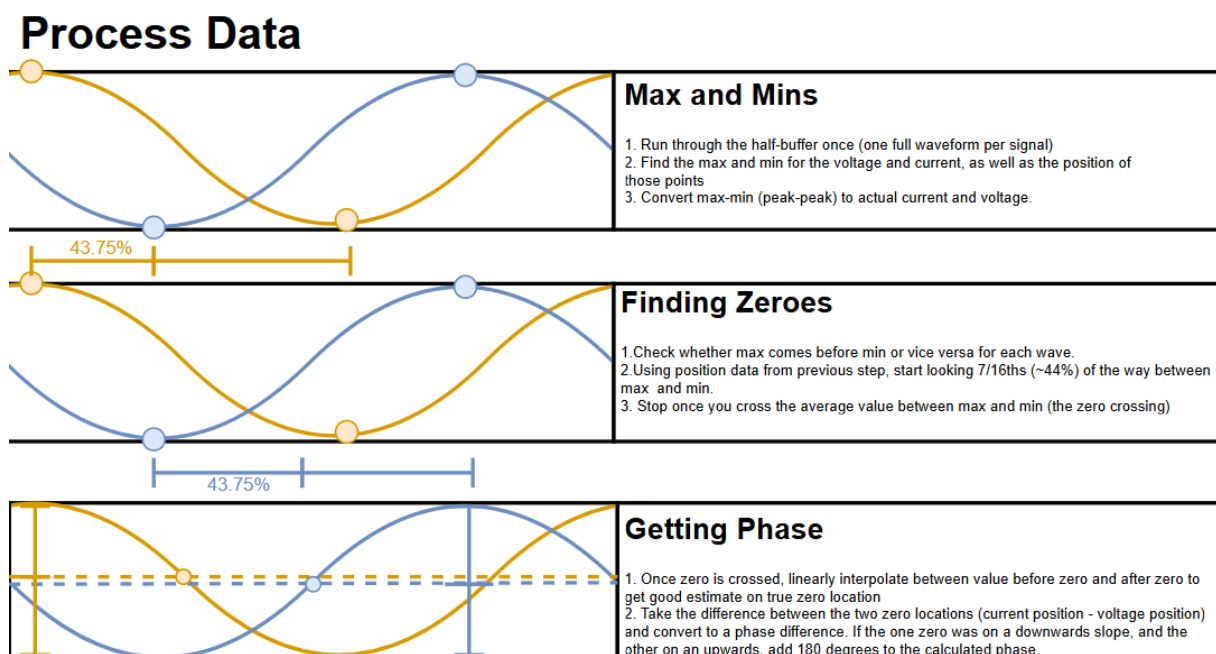


Figure 15: Steps to process data

The read voltage and current can be converted to a real world value using the following formula, where the calibrated max and min are hard-coded values representing the maximum and minimum values the ADC gave when 2.2VPP (for the voltage) and 2VPP (for the current) was applied. These

constants were determined experimentally, using 3 different signal generators:

$$V_{real} = \frac{\text{Max}_{meas} - \text{Min}_{meas}}{\text{Max}_{calibrated} - \text{Min}_{calibrated}} \cdot 253V, \quad I_{real} = \frac{\text{Max}_{meas} - \text{Min}_{meas}}{\text{Max}_{calibrated} - \text{Min}_{calibrated}} \cdot 15\,000\text{ mA}$$

There is slight error that arises from this equation, as it assumes every step up until this point has been a linear conversion. Any non-linearities in our op-amp or ADC, and the equation will deviate.

There is some internal scaling and moving of data indexes, but the main idea of the phase is worked out using the following equation, from the data discussed in Figure 15.

$$\text{Phase} = \theta_v - \theta_i = [\text{pos}(\text{current zero}) - \text{pos}(\text{voltage zero})] \cdot \frac{2\pi}{n_{\text{samples per wave}}}$$

The subtraction seems backwards because, as the phase angle of a given wave increases, the zero moves further left (aka the position decreases).

The phase value itself is clamped to $[-\pi; \pi]$ radians, though in reality the end user will almost always be within the range $[-\frac{\pi}{2}; \frac{\pi}{2}]$ - any further and the user would be putting power back into the grid.

The calculated voltage, current and phase are saved globally. They are also added to accumulator variables, that add up a running sum of voltage, current, phase, and power (see section 4.4 on this calculation). The average power is used to decrease units. It is valid to use the mean power like this to work out power over time, as long as the time interval between power calculations (t) is constant:

$$\text{Energy Consumed over } n \text{ samples} = \sum_{i=0}^n P_i \cdot t_i \xrightarrow{t \text{ constant}} t \sum_{i=0}^n P_i \xrightarrow{\text{total period } T=nt} T \frac{\sum_{i=0}^n P_i}{n} = T \cdot P_{avg}$$

The other values (V, I, phase) are accumulated to work out their average over the last second, to provide a more accurate log - this is done in section 4.9.

4.4 Power calculations, P, S, Q, and PF

In order to work out the energy the end user is using over time (UR4), we must know the power that they are drawing from the wall at any one time. This is an easy calculation to work out, now that we have the current voltage, current, and phase information of our two measured waves. Because we know the signals are AC (as in, steady-state sinusoids), we can make use of the complex power equations:

letting $V = V_{rms}$, $I = I_{rms}$, and φ = phase difference between the two

$$S = V \cdot I = \text{Apparent Power}$$

$$P = V \cdot I \cdot \cos(\varphi) = \text{Real Power}$$

$$Q = V \cdot I \cdot \sin(\varphi) = \text{Reactive Power}$$

$$\text{Power Factor} = \cos(\varphi)$$

These equations are used as-needed whenever the user wants to see real-time information about their power usage (E.G. during a stat command or on the real-time menu). Doing so means that one only has to store the most recent voltage, current and phase - every other quantity is re-derived as needed.

Real power is always kept positive, as the end user will likely only ever be drawing power, not sending it to the grid, but the system has been designed to handle negative real power (e.g. it will accurately report the phase of a signal with negative real power).

Table 3: Valid UART Commands and Responses

Command Format	Description	Response	Instant
On Startup	Transmit student number	*27005348#\n	N/A
*Stat#\n	Request SB status information	Covered in section -	
*Load#\n	Toggle load switch ON/OFF	Covered in section -	✓
*Log#\n	Toggle logging to SD Card ON/OFF	Covered in section -	
*Dump#\n	Dump SD card file contents over UART to PC/TS	Covered in section -	
*CLF#\n	Clear File to erase SD card data	Covered in section -	✓
*UnitsXXXXX#\n	Adds XXXXX pre-paid units (Wh) to the system, where XXXXX is an integer from 00000 to 99999	Covered in section -	✓

4.5 UART communications

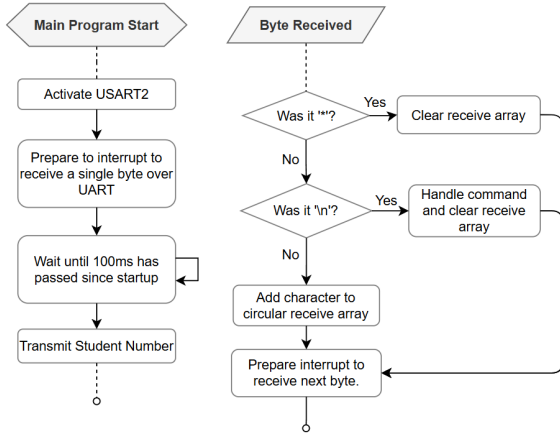


Figure 16: UART Flow Diagram

To better facilitate debugging and administrative control, the system makes use of a UART interface for sending and receiving data to and from another computer. This is specified in [UR2](#).

The UART interface is configured with a baud rate of 57600 bit/s, in an 8E1 configuration. This gives an effective bandwidth of

$$\frac{57600}{8 \text{ data} + 1 \text{ start} + 1 \text{ stop} + 1 \text{ parity}} \approx 5236 \text{ byte/s}$$

All transmissions are handled using the HAL layer's default UART Transmit function, while receptions are handled on a per-byte basis, building up the received command one byte at a time. This uses the interrupt version of the HAL UART receive function, allowing us to stop the program to run a callback function each time a byte is received.

I have done this as we are expecting variable-length input messages, which the HAL layer is not designed to deal with easily. In order to not miss a message, the byte received callback needs to be as lightweight as possible, so that it can run to completion before the next byte is received.

The code first checks if the received byte is '*', indicating the start of a message. If it is, it just clears the array that keeps track of the received message. If the byte was instead a newline character, the function sends the message off to be immediately processed. This is done in the interrupt and not in the main loop to ensure that the program properly catches the full message, before even preparing the next byte to be read. If it was in the main loop, there is a chance that the message buffer gets overwritten before it is properly processed. The only downside to this approach is that two commands cannot be sent sequentially (without a pause). This was deemed appropriate, as no user requirement indicated this level of responsiveness from the system. Critical commands are handled immediately after processing, while commands that take longer to run are offloaded to be completed within the main loop.

If the byte was anything else, it simply gets added to the circular message buffer - The buffer wraps around after 16 characters (the longest valid message only needs 11), which means that even if junk data is received, the buffer will never overflow. The '*' check ensures that valid messages will always begin at the start of this buffer, so it is fine to haphazardly store every byte received.

4.6 Keypad driver setup and button bounce handling

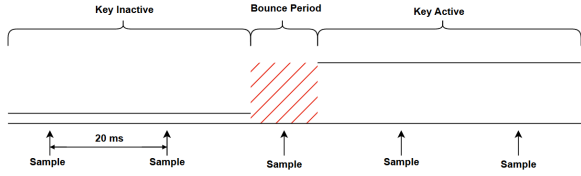


Figure 17: Keypad Debouncing

Both the button and keypad provide physical input for the end user to interact with our system. In order for the system to feel responsive, the state of the button and keypad must update within software fairly rapidly. I took two different approaches at solving this problem:

The GPIO pin that the button is connected to is linked to an external interrupt - this lets the system immediately know that the button has been pressed.

The interrupt sets a flag to let the system know that a button has been pressed, and stores the time of that press. The system can then check 25ms later if the button is still pressed, allowing the system to ignore any bounces that might occur.

The keypad is handled differently: Instead of trying to use interrupts to deal with 12 possible key presses, I check which keys are being pressed on the keypad every ≈ 25 ms. Figure 17 demonstrates how this is equivalent to debouncing the signal: as long as the sample rate is slower than the bounce period, the system will only read within the bounce period once. This is fine, as either a low is read, and the keypad only gets it right on the next sample, or a high signal is read, and the keypad correctly registers that a key is being pressed during this sample period - there is no way to accidentally read two key presses by doing this.

This method has the added benefit that the system can detect multiple key presses at once, or even new key presses while old keys are still being held. A more in-depth explanation of the keypad and button drivers is shown in Figure 18.

4.7 OLED low-level interface

The OLED display is powered by an SSD1306 controller. This is a popular controller for many OLED displays, and many drivers have been written for it. I am using RobertoBenjami's [14]. It communicates over I2C, and has DMA compatibility. This allows us to offload about half of the processing time to the DMA controller. Side note - referencing this repo does not work correctly - the underscore break latex.

The driver works by first building up the image buffer to send to the screen in memory, through a host of draw commands, and then sends that internal buffer to the screen over DMA once we tell the driver to update the screen. The driver supports continuous updates, but this lead to strange visual artifacting, so I chose not to activate it.

We are able to use DMA for this task as the microcontroller has 2 DMA controllers[8], so we never have contention with the ADC's DMA requests. The driver also only works in fast I2C mode, at 400kHz, which was helpful, as this quartered the amount of time used to transmit the display buffer.

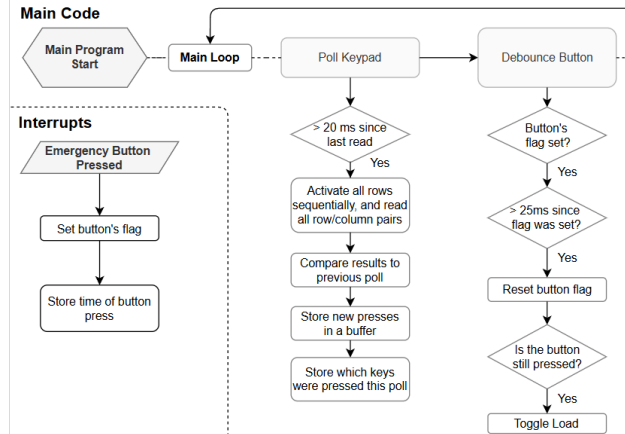


Figure 18: Keypad and Button Software Handling

4.8 FSM control and logic

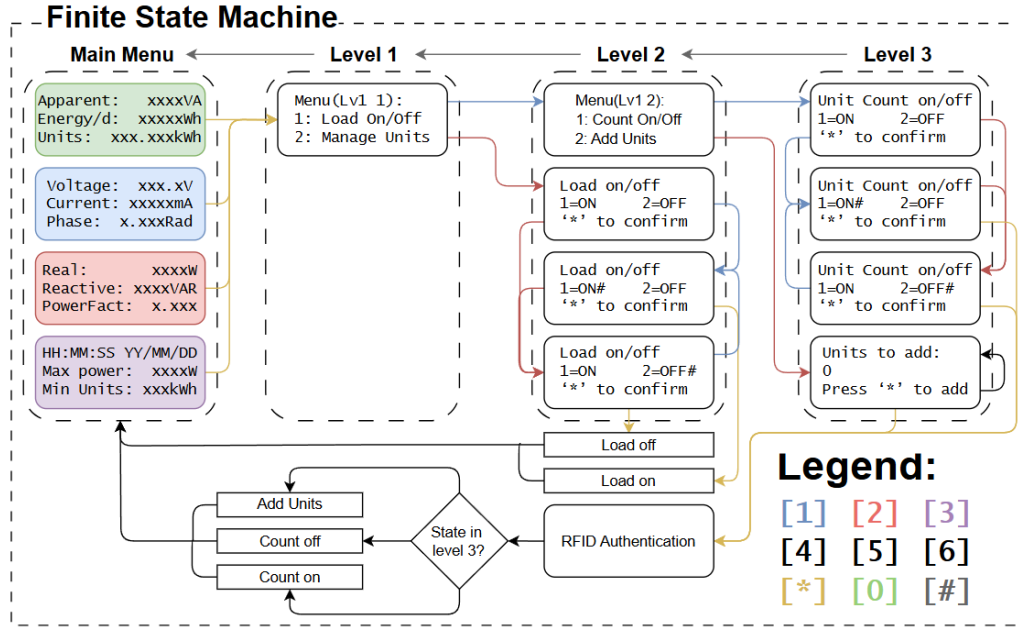


Figure 19: Finite State Machine States and Transitions

In order to employ a user menu with persistent state, the system incorporates a finite state machine (FSM) to manage what is currently displayed, and what each key is able to do at any moment. This is shown in Figure 19. Each key press has been color-coded, and shown in the legend.

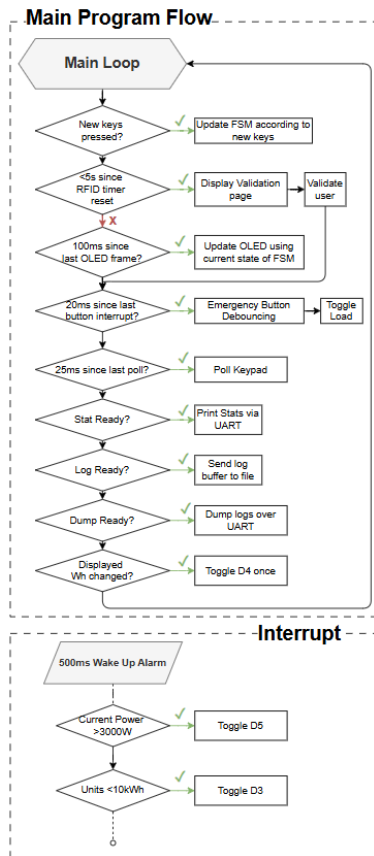


Figure 20: Main Loop and Interrupt Control

The state machine I employ is a little unconventional. Instead of a single state variable that represents each screen, I use an array of states with 4 entries. Each spot in the state array represents the state within a certain level (e.g. slot 0 represents which of the main menu pages are open, slot 1 the level 1 pages, etc.) This allows me to go back to the previous page just by moving back one level in the state array.

This method also enables a trick during RFID authentication: Instead of saving a variable saying what I am authenticating the user to do, I just look at what state slot 3 had last - this lets me know which page I just came from. It is also how I made RFID authentication from the UART units command work: I just changed the slot 3 state to be the unit page before resetting the RFID timer. The complete main loop is shown in figure 20. Most of it is self-explanatory, and just included to show the full extent of the main loop.

Of particular note is what the RFID check does to the OLED - only if the RFID timer is greater than 5s does the OLED update to the current screen. This was done because, as the code is written currently, the screen can only change state whenever a key is pressed - this is at odds with the 5s timeout of the authentication screen - therefore, the RFID had to bootstrap the FSM display functionality in order to change back to the default page after authentication succeeds or times out.

4.9 Energy consumption calculations and RTC

UR11 requires that our system use a real-time clock. The main driving force behind the need for one is to accurately keep track of how much energy the end user is using over time.

I use the RTC to both work out how much power the user has used, and to accurately log this information to the SD card once a second. The RTC is configured to start at 2025/02/26 22:12:42, just for testing purposes.

In order to accurately measure power, and to log exactly once a second, I have a wake-up alarm set in my RTC at 2Hz, or every 500ms. This is a feature that allows for sub one-second interrupts, and is linked directly to the RTC's clock (A low-frequency quartz oscillator tuned to be almost exactly 32768.0Hz - 2^{15} Hz). In order to get to 500ms, I had to work out the correct scaling and pre-scaler values. The calculations for this are identical to the ones used for the ADC, with the slight caveat that the prescaler is not a register value that we can arbitrarily set - it has fixed options, and so there is no need to add 1 (to accommodate for the possible PSC = 0 case).

$$f_{\text{out}} = \frac{f_{\text{clk}}}{\text{PSC} \cdot (\text{ARR}+1)} \Rightarrow \text{PSC}_{\text{max}} = 16 \Rightarrow 2 = \frac{32768}{16 \cdot (\text{ARR}+1)}$$

This gives an auto-reload value of 1023. The alarm is running at 500ms as a quirk of another feature: the Wh ticker. Section 4.8 briefly covers the behaviour of this LED, but the main point is that D4 must toggle once per Wh used. This means that our system must update the units fast enough to capture exactly 1 Wh of change. At a maximum rated voltage of 253V and maximum current of 15A, the greatest amount of power the system can measure at a point is $253 \cdot 15 = 3795\text{W}$. If we updated this once a second, this would be equivalent to $\frac{3795}{60 \cdot 60} \approx 1.05\text{Wh}$ of energy used in one second. This means that there is a possibility for the system to reflect a 2Wh change in a single update. To rectify this, I chose to update the units every half-second. Using the average power discussed in section _ for the power calculations, I worked out the one-second conversion from real power to Wh units using the following equation:

$$X \text{ Wh} \cdot 60 \frac{\text{min}}{\text{hour}} \cdot 60 \frac{\text{seconds}}{\text{min}} = Y \text{ joules of energy}$$
$$Y \text{ joules} = Z \text{ Watts} \cdot N \text{ Seconds} \Rightarrow \therefore X \text{ Wh} = \frac{Z \text{ Watts} \cdot N \text{ seconds}}{60 \cdot 60}$$

Each half second, I subtract $\frac{Z \text{ Watts} \cdot 0.5}{60 \cdot 60}$ from the total units stored. This value Z is the average power over that half second, mentioned in section 4.3.

Halving the alarm's period allowed me to use the same alarm to log data every two callbacks (once a second). Using an alarm linked directly to the RTC like this ensures that each log in the SD card is exactly one second apart (from the system's point of view), ensuring that the same time is never double-logged.

4.10 SD Card Interface and Logging

The SD card, as described in section 3.8, uses an SPI interface. To be able to communicate with it, I am using kiwih's SPI SD Card driver [15], which interfaces with the FATFS library. FATFS is an open-source library for microcontrollers that enables FAT support (a type of file system), and is built in as supported middleware in the STM32CubeIDE. kiwih's library enables FATFS to talk to our SD card over SPI, while FATFS handles all the abstraction on how file handling is managed. An example log is shown below. At worst, it is 47 characters long.

2025/04/29 17:05:58,246.7,13001,(-)0.017,010.008\n

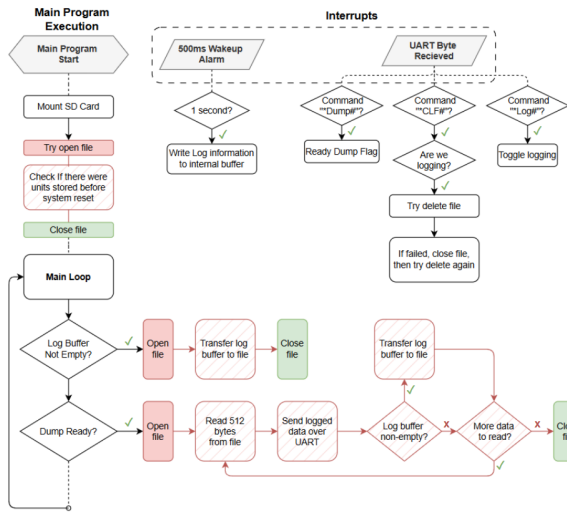


Figure 21: SD Card Flow Diagram

Dump reads in 512 byte chunks, as the SD card is faster at sequential reads than instantaneous ones (REF), speeding up the already slow dump process. At the calculated bandwidth of 5236 byte/s over UART, the dump command takes $\frac{47 \text{ characters}}{5236} \approx 9$ milliseconds per log, even ignoring read times. At just under two minutes of logs, dump will take a second to print.

4.11 RFID Interface and authentication

The RFID reader uses the SPI interface, just like the SD card reader. I have used CircuitGatorHQ's [16] driver for this purpose. It uses SPI to talk to the RFID reader. The setup of this driver is detailed in CircuitGatorHQ's Youtube video as well as README files.

The RFID authentication system is used whenever administrative access is needed within the system: to be able to turn on and off the units counting down, and to add new units to the system. Whenever any of these features are run in code, I route them through the RFID authentication portion of my code. It works as follows:

The request for administrative action is sent by the FSM. This consists of a reset of the RFID timer while being on the correct menu page. The next time the main loop runs, it sees the RFID timer has been reset, and hijacks the screen to show the authentication screen. While on this screen, I run a simple state machine that attempts to:

1. Request information from all nearby idle cards (using ReqIDL command)
2. Obtain the RFID UID and checksum byte from a single RFID tag (Using the anticollision command)
3. Select the tag with the given UID - getting it to send how much space it has available
4. Check that the target's UID is an authentic user

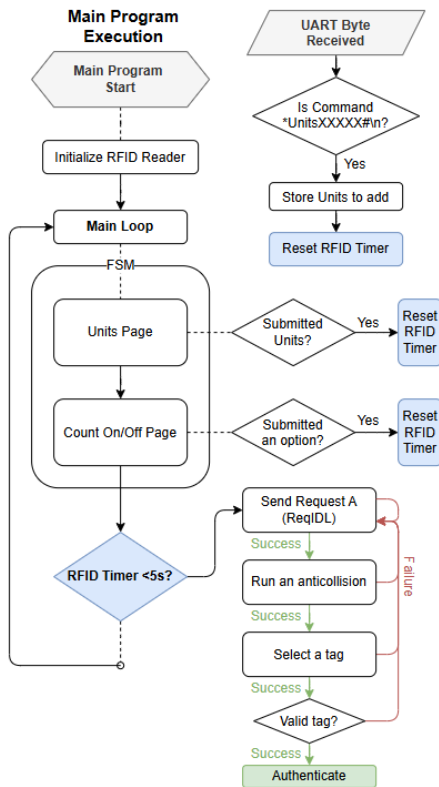


Figure 22: RFID Authentication Process

Each time a request fails, I reset the state back to the idle request, let the main loop run once, then try again. This allows the RFID authentication to be non-blocking.

This process is slightly overengineered - step 3 is not necessary if we just care about reading the UID, but it is a holdover from when I was trying to authenticate new users on-device.

If the authentication is successful, I check what state the FSM was in before the timer was called, and run the corresponding routine. This system is also how I manage to authenticate the units UART command - I manually change the state of the FSM to the add unit state, reset the RFID timer, and resume from there.

The system automatically returns to normal once the timer runs out, as the RFID system no longer takes over the display if the timer is past 5 seconds.

5 Measurements and Results

5.1 Power Supply

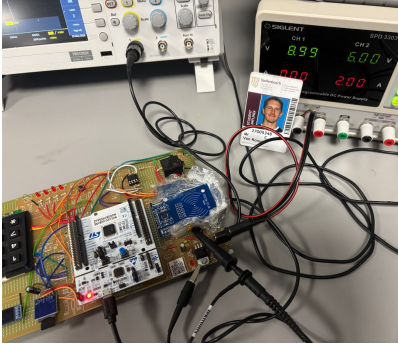


Figure 23: Setup for measuring 3.3V Regulator

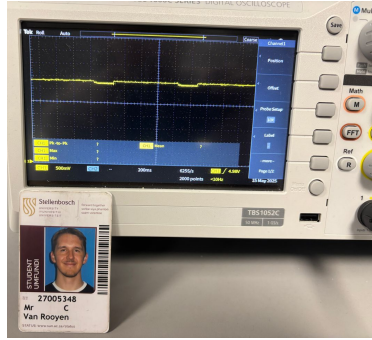


Figure 24: 3.3V Maximally Loaded

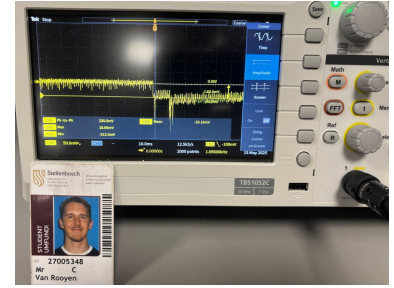


Figure 25: 3.3V Maximally Loaded (Zoomed)

In order to test the power supply, I attached my system to a DC signal generator, and ran the system at a host of voltages and load conditions, viewing the output voltage level from the 3.3V and 5V regulator on an oscilloscope. My tests are as follows:

I tested if the system could handle $\pm 1V$ over the designed-for voltage range (9-12V), while under load conditions. The testing setup for this is shown on figure 23, for a test of the 3.3V regulator.

PSU	8V, Variation	8V, DC	13V, Variation	13V, DC
5V	+22, -16mV	5V	+20, -18mV	5V
3.3V	+24, -40mV	3.3V	+24, -40mV	3.3V

We expect no real issues here, as the dropout voltage for the 5V power supply is 2.5V, and the maximum input voltage is 35V.

The above results demonstrate that, under load conditions, our system is able to handle the full range of expected input voltages.

Figure 24 and 25 show my testing of the 3.3V line under full load conditions - when the logging and RFID are active, as well as all other passive peripherals. In the worst case, the 3.3V line drops by $\approx 60mV$. This is within the 5% tolerance allowed by the system specifications ($3.3V \cdot 5\% = 165mV$)

I also tested that the power supply did not work when the input rails wires were backwards by swapping the wires shown in figure 23 that are connected to the DC power supply. I tested up to 16V in reverse, and the power supply sent no current.

5.2 V and I circuits



Figure 26: Op-Amp Output at Max Inputs



Figure 27: Op-Amp Output Zoomed (V=yellow, I=blue)

I tested for two things with the VI measurement circuitry: My software calculations rely on the assumption that our V and I measurement conditioning circuitry is completely linear (within the region we are measuring). This means no clipping from the output of the op-amp, and a linear transfer function from the input to output of the op-amp. The specific region we are targeting is between 0 and 3.3V, as this is the range of our ADC.

Figure 26 demonstrates the testing I did to test when clipping occurred. Firstly, at maximum inputs (2.2Vpp for V and 2.0Vpp for I), neither output clipped. The voltage waveform ranged from [0.14V; 3.14V], while the current waveform ranged from [0.24V; 3.2V].

This leaves some leeway for possible offset in the input signal, if it is ever accidentally not centered at 0V. Increasing the voltage now, I found that the voltage waveform shows clear clipping at ≈ 2.5 Vpp input, and the current waveform clips at ≈ 2.2 Vpp.

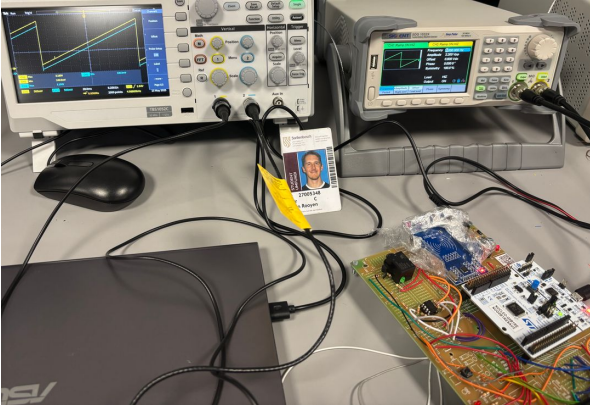


Figure 28: Testing Op-Amp Linearity



Figure 29: Op-Amp Output Zoomed (V=yellow, I=blue)

Figure 28 shows the testing I performed to verify the linearity of the op-amp circuit. I tested this by inputting a triangle wave (each with the same maximum amplitude as the previous test) into each of the V and I circuit's inputs. In the worst case, we would observe something a non-linear response coming out of the op-amp. This would have to be corrected in software, before being able to be converted to the real-world values. Fortunately, figure 29 demonstrates that, for all intents and purposes, the op-amps's response is linear - we get a scaled and shifted triangle wave back from the op-amp. We can still account for any marginal nonlinearities, but I elected not to as my measurements were already within tolerance.

5.3 V, I, Phase, P, S, Q

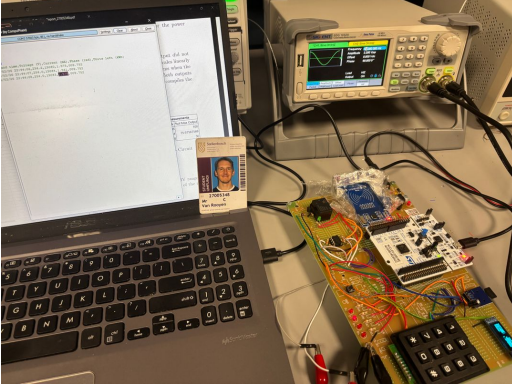


Figure 30: Testing Setup for Signal Generator 1

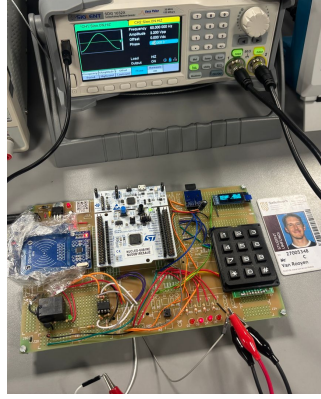


Figure 31: Testing Setup for Signal Generator 2

Testing of the measurement circuitry was conducted after all the components were soldered to the board. This was done to provide results that are as accurate to the normal operation of the board as possible. Figure 30 demonstrates the setup used to perform all the tests in this section. A few steps were taken to minimize the measurement errors. Firstly, all oscilloscope measurements were made using oscilloscope probes.

Secondly, I calibrated to multiple signal generators, two of which are shown here. Lastly, I used the logging feature of the system in order to get measurements on the average reading over one second - this eliminated random noise from the results.

I took a range of values for voltage, current, and phase, and set them on the signal generator, then logged the data, and wrote down what the logs gave. These results are shown in Table 4, and the data that produced them is included in the appendix.

Phase Values			%Error		
Real (rad/s)	Signal Gen 1	Signal Gen 2	Signal Gen 1	Signal Gen 2	
1.570796327	1.575	1.572	0.2676141479	0.07662821618	
1.178097245	1.17		0.6873155107		
0.7853981634	0.788	0.783	0.3312761251	0.3053436472	
0.3926990817	0.395		0.5859240341		
0	0	-0.02	N/A	N/A	
-0.3926990817	-0.394		0.3312761251		
-0.7853981634	-0.78	-0.781	0.6873155107	0.5599915562	
-1.178097245	-1.178		0.008254420132		
-1.570796327	-1.575	-1.574	0.2676141479	0.2039521707	

Voltage Values			%Error		
Real (V)	Signal Gen 1	Signal Gen 2	Signal Gen 1	Signal Gen 2	
0	0	0	N/A	N/A	
207	206.8	207.4	0.09661835749	0.193236715	
230	230.5	231.5	0.2173913043	0.652173913	
253	254.1	254	0.4347826087	0.395256917	

Current Values			%Error		
Real (mA)	Signal Gen 1	Signal Gen 2	Signal Gen 1	Signal Gen 2	
0	0	0	N/A	N/A	
1500	1516	1520	1.066666667	1.333333333	
3000	3016	3012	0.5333333333	0.4	
4500	4503	4507	0.0666666667	0.1555555556	
6000	6007	6013	0.1166666667	0.2166666667	
7500	7514	7527	0.1866666667	0.36	
9000	9002	9006	0.02222222222	0.0666666667	
10500	10493	10505	0.0666666667	0.04761904762	
12000	11997	12017	0.025	0.1416666667	
13500	13505	13500	0.03703703704	0	
15000	15045	15078	0.3	0.52	

Table 4: V, I, and Phase Results

I tested phase in steps of $\frac{\pi}{8}$, in the range $[-\frac{\pi}{2}; \frac{\pi}{2}]$, as this is the range the end user is most likely to lie in. The same logic dictated my choice of voltage measurements - I tested with the voltage off, at the expected 230V, at 230+10%, and at 230-10% - the full range of expected voltage values. Current was a little different, as the user's current can be anywhere from 0 to 15A. I chose to test in 10% steps, to try capture the full range of values.

The percentage error has been worked out the same way as mentioned in section 5.1. It is not entirely accurate for phase, as we are allowed an absolute error of 5%, and not a relative one, but the fact that all the errors are under a relative 5% error anyway is an even better result. These results are surprisingly accurate, and validate that our assumptions about our measurement's linearity did not introduce enough error to stray from the 5% tolerances given to us.

An interesting way we can test the system, albeit challenging to work out precisely what the exact real world-answer should be, is to slowly linearly increase the amplitude/phase of our signals. This can be done using an amplitude modulated signal with a very slow frequency (in the case of changing amplitude) or a phase modulated signal with a very low frequency (for the case of varying phase). Combining this with the logging ability of our system allows us to easily automate the testing process over the whole range of input values. These results are shown below, and show that the system is capable of reading the complete range of amplitude and phase values linearly. Again, the data that produced these graphs is included in the appendix.

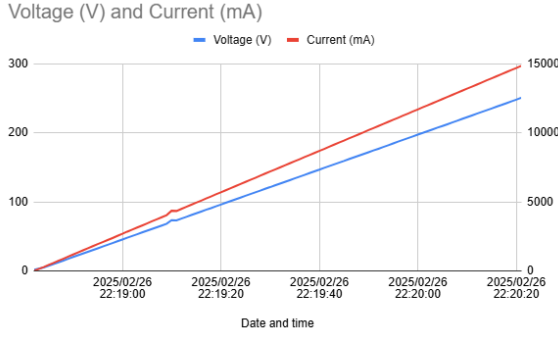


Figure 32: Voltage and Current Sweep Over Full Range

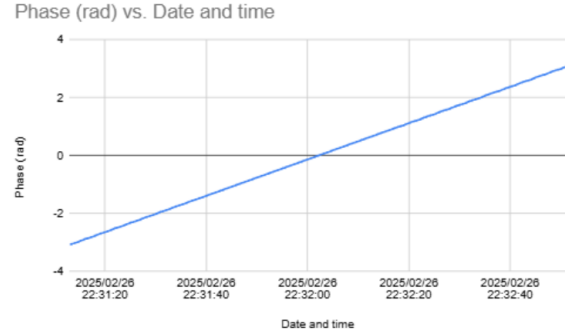


Figure 33: Phase Sweep Over Full Range

5.4 Load Circuit

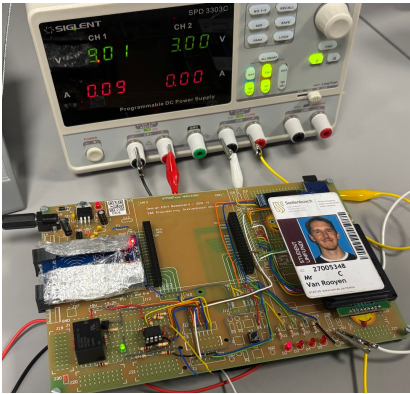


Figure 34: Minimum Mosfet Input Test

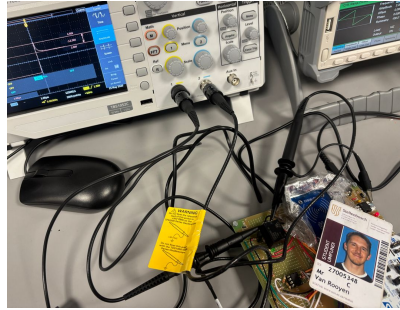


Figure 35: Verifying Flyback Diode behavior

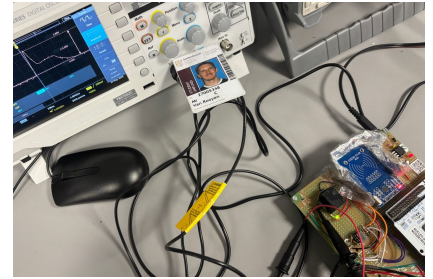


Figure 36: Testing Inrush Resistor

I conducted 3 tests on the load circuit: Firstly, was the mosfet able to switch the load with non-ideal input voltage? Secondly, was the flyback diode behaving as expected? Third: Was the inrush resistor performing its function? These are shown in figures 34, 35, and 36 respectively.

For test 1, I removed the STM32 board from the PCB, and manually applied a DC voltage at the load control, then tried to give the load the lowest possible voltage where it would still turn on. This was at 3V - giving the STM32 board a 10% margin on its 3.3V output before being unable to turn on the load.

In test 2, I put a probe on either side of the flyback diode, then turned off the load, to graph the voltage over the diode. The voltage spikes up to 4.56V, then drops to the diode's turn-on voltage of 2V. This indicates that the diode is working: Diodes have an exponential relationship between voltage and current after their turn-on voltage, and so a spike in voltage like this indicates that a large amount of current is rushing through the diode whenever the relay is turned off.

Test 3 was a similar setup, except the probes were over the inrush resistor. I turned on the load, charging up the gate capacitance of the mosfet, and measured the voltage over the inrush resistor. This is equivalent to measuring a scaled version of the current out of the GPIO pin. As predicted by the theory, the highest current occurs at the very beginning, when the capacitor is empty. This current occurs at a peak of 1.90V. Through a 1K Ω resistor, this is a current of 1.9mA. This is better than the theoretical worst case 3.3mA we designed for.

5.5 Keypad and Button

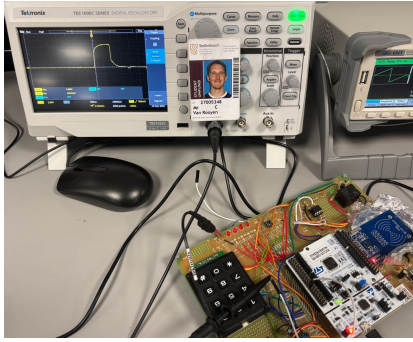


Figure 37: Keypad Bounce Test

I tested the keypad for any bouncing in order to get an estimate on how often I should be polling it. One such test is shown in Figure 37. Unfortunately, after 100 presses, I was unable to catch any bounces. The style of keypad (using force-sensitive resistors) is particularly tolerant against bouncing. The reason for this can be seen the same figure: Soft presses, like the one shown, smoothly raise the voltage up to 3.3V when reading the keypad. Harsh presses, like the ones that normally create bouncing, require too much force for the button to be able to bounce fully back up and cause a misread.

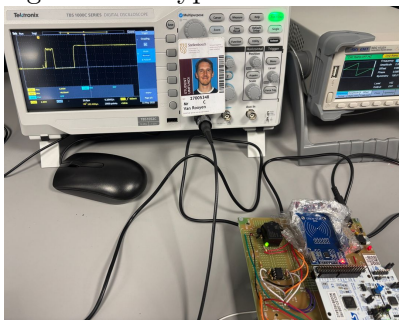


Figure 38: Button Bounce Test

I repeated the same bounce test for the emergency button. This time, I did see bouncing occurring. The particular test shown in figure 38 was a particularly long bounce - about 100 μ s. This is far faster than what a human could produce, but is slow enough that the STM32 board would read this as two key presses.

Choosing a debounce time of 25ms was more of a conservative decision than one based on hard evidence - It is far faster than what a human could reasonably do to a button, but slow enough to not register any possible bouncing.

5.6 UART Communications



Figure 39: UART Transmit

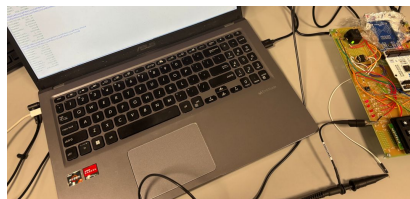


Figure 40: Two Commands, No break in Receiving

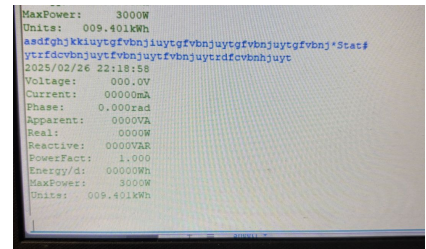


Figure 41: Command In Middle of UART Message

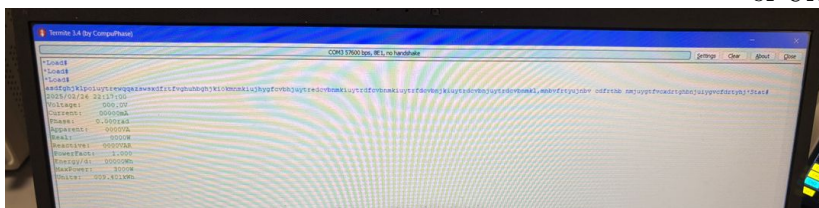


Figure 42: Really Long Message with Command

To test the UART communication, I ran the system through a host of tests - a few of the most important ones have been included in figures 39 to 42.

Figure 39 demonstrates the board's ability to transmit over UART by sending my student number. I viewed this output on the oscilloscope by plugging the probe into the test stations SB tx pin.

Figure 40 demonstrates the board's ability to continuously receive and process UART commands, without breaks. I sent a single long message, that composed of ≈ 20 characters, followed by a stat command, followed by more characters, followed by a load command. The test showed that both stat and load were processed and ran, thus proving the UART's resilience against junk messages.

Figure 41 was similar, except it just used a single real command amongst a lot of junk data. Again the UART handling was resilient enough to process this command.

Lastly, Figure 42 demonstrates the board's ability to deal with arbitrarily long transmits - I sent a junk command with over 50 characters to the UART, with the real command at the very end, and again the protocol was able to successfully receive and act upon the stat command.

5.7 OLED and Menu Functionality

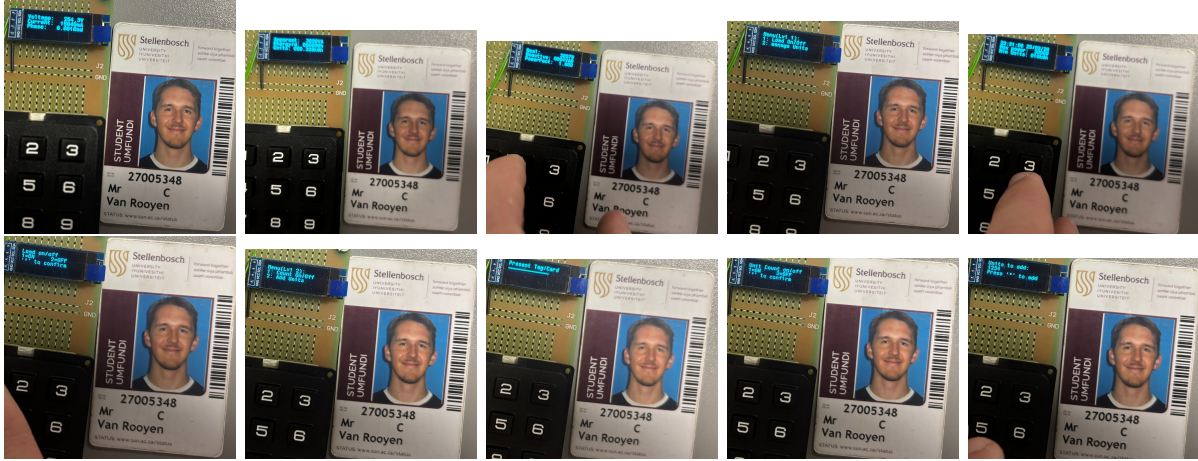


Figure 43: All OLED Screens

Figure 43 demonstrates every possible OLED screen that the system can show - this shows that the OLED is functional, as well as the FSM that controls the state logic behind it.

5.8 SD Card

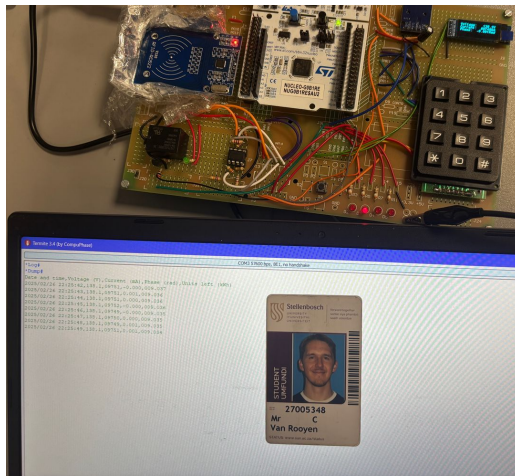


Figure 44: SD Card Access (Logging)



Figure 45: Timing Logging Rate

To prove that SD card functionality was working, I used the logging feature to store and recall multiple seconds worth measured data. Figure 44 shows a log over 7 seconds. The most recent log agrees closely with the instantaneous voltage, current, and phase reported by the screen, indicating that successful recall from the SD card has occurred. The voltage, current and phase reported by the OLED are 138.2V, 9749mA, and -0.001 Rad, if it is not clear to read off the figure.

Figure 45 demonstrates how close to reality the logging rate to the SD card is - in 10.45 seconds, the SD card stored 11 samples. This is within tolerance, as 11 samples is correct if the RTC was more than 55ms through a second when logging was turned on.



Figure 46: RFID Authentication Screen



Figure 47: RFID accessing Tag's UID

5.9 RFID

The above two figures demonstrate that RFID functionality is working - Figure 46 shows that the authentication screen is working, and Figure 47 is a snapshot of the debug view, showing that the RFID reader has successfully obtained the tag's 4 byte UID + checksum byte. My blue tag's ID is {197;42;177;1;95}.

5.10 Full System

To be able to validate that the system functions correctly, I stress-tested the system in an effort to confirm its reliability and consistency.

Focusing on the consistency aspect - throughout the whole testing process, the system never gave any erratic responses to any of the given input channels: UART always responded the same, the emergency load control always toggled the load, each keypad press always did the same thing, etc. The only aspect of the system that is not fully consistent is the measurements, which vary slightly from one signal generator to another. This was covered in section 5.8. UART used to skip every few commands earlier on in the design cycle, but this issue was resolved with a reordering of interrupt priorities, combined with offloading more of the UART interrupt's work to the main loop. The file handling also used to be far more inconsistent, when everything was on interrupts, as concurrency issues were a big problem, but that was fixed once all file handling was moved to lie within the main loop (except CLF).

In order to test reliability, I tried to run as many of the measurements shown in this section without ever resetting the system. Not once did the system fail or freeze up. There seems to be an issue with the way I am using RFID, however. Each authentication event, the time necessary to validate a card gets longer and longer (but it does plateau to about 3s). I have some theories as to why this is, but I have saved the discussion for the conclusion.

The other reliability test I did was to run the system overnight. By the time I had woken up, the system was as responsive as it was 7 hours ago - this is a strong indication that there are no memory leaks, integer overflows, or similar bugs that would appear as the system ran for a long time.

6 Conclusions

6.1 System Compliance/Non-Compliance

The system that I have made is feature-complete. It has every feature that user requirements and system specifications state as needed. That does not mean it is without its quirks.

One such quirk is the RFID behavior - the RFID was introduced too late into the system's development for it to be seamlessly integrated with the rest of the codebase, and so it has been bootstrapped onto a number of systems. For example, the RFID takes over the FSM's job of writing to the screen while the RFID timer is counting to 5, but this does not pause the FSM. Therefore, you can still navigate menus and perform actions while the authentication screen is active.

Another such quirk was introduced when the SD card was added - because the SD card draws so much current, and the 3.3V regulator can only supply about 100mA, Each read and write of the SD card causes the 3.3V line to drop considerably, harming the ADC's ability to read against a proper 3.3V reference voltage. I haphazardly solved this by pausing and unpausing the ADC's timer, but this would not be a permanent solution if this was a real device that we were making. Preferably, I would've added a separate, isolated 3.3V power supply just for the ADC, so the components less sensitive to changes to this line's voltage wouldn't mess with the components that did care.

6.2 Improvements to be Made

There are a lot of changes I would've made to the system had I had the time: for one, I had no idea about the RTC's wake-up timer before demo 4. I was trying to link the RTC's clock to the ADC in order to get the most accurate sampling of the waves possible all the way back in demo 2, but I had no way of subdividing the RTC past a millisecond (is what I thought).

Secondly, I should have done a proper calibration analysis on the whole measurement process - The ADC does have some non-linearities that can be accounted for, they just aren't immediately visible when looking at the full range of the ADC all at once.

Third, there were some better hardware decisions I could've made - the foil on the RFID reader, while effective, also significantly reduces the effective range of the device. Had I put the RFID read further to the exterior of the device, and used less angled foil, the system would be much more forgiving to the end user.

Also, reiterating the point about the 3.3V line - sharing it between so many problematic devices created a large headache in the design process, and it would've been easier to have separate lines to isolate each device.

There are many edge cases in code that I know will cause the system to act erratically - for example, taking out the SD mid-operation or sending two consecutive UART commands without pause. I had a solution in mind for most of these, but I just ran out of time. Like for the RFID reader - I had tracked the cause of the slowdown to either be because the MFRC552's internal buffer was filling up, and needed to be emptied, or because I was calling ReqIDL but the card was already awake (and I only discovered ReqAll very recently).

6.3 Final Thoughts

All this complaining aside, the system still follows the complete specifications given to it. And for that, I am proud of what I have achieved.

References

- [1] *LM340 LM340A and LM7805 Family Wide VIN 1-5 A Fixed Voltage Regulators*, Texas Instruments.
- [2] B. Carter, *Designing Gain and Offset in Thirty Seconds*, 2002.
- [3] *12-Button Keypad*, SparkFun Electronics.
- [4] *L7800 Series Positive Voltage Regulators*, STMicroelectronics.
- [5] *L78LxxAB L78LxxAC - L78LxxC Positive Voltage Regulators*, STMicroelectronics.
- [6] *MCP601/1R/2/3/4 Datasheet. 2.7V to 6.0V Single Supply CMOS Op Amps*, Microchip Technology Inc, 2007.
- [7] *3.0mm Round Type LED Lamp*, BrightTEK.
- [8] *STM32G0B1xB/xC/xE Datasheet, ST Microelectronics, Rev 5, June 2024*, STMicroelectronics.
- [9] *Design E314 2025 PDD V0.9d*, Electrical and Electronic Engineering - Stellenbosch University.
- [10] *SANS 10142-1*, South African Bureau of Standards, 08 2024.
- [11] *J107 Series Relay/Switch Datasheet*, STMicroelectronics.
- [12] *STM32G0B1xB/xC/xE Datasheet*, STMicroelectronics.
- [13] *Advance Information SSD1306 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*, Solomon Systech.
- [14] RobertoBenjami, “stm32-ssd1306-i2c-dma-hal – github repository,” <https://github.com/RobertoBenjami/stm32-ssd1306-i2c-dma-hal>, 2023, accessed: May 18, 2025.
- [15] kiwih, “cubeide-sd-card – github repository,” <https://github.com/kiwih/cubeide-sd-card/tree/master>, 2020, accessed: May 18, 2025.
- [16] CircuitGatorHQ, “RFID-RC522-with-STM32 – github repository,” <https://github.com/CircuitGatorHQ/RFID-RC522-with-STM32>, 2025, accessed: May 18, 2025.

Appendix

Pinout

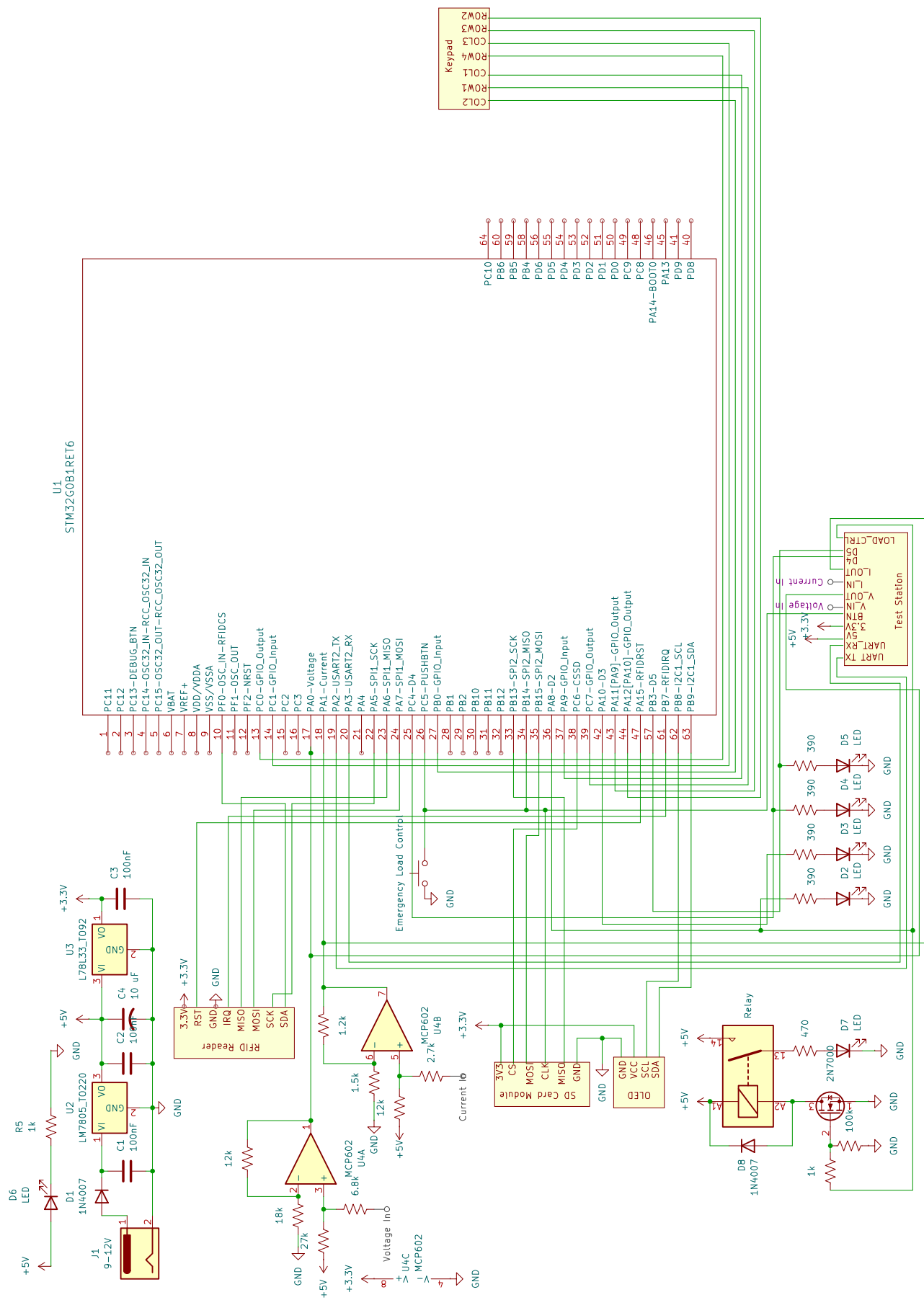
Name	Type	Signal	Label
PC13	I/O	GPIO_EXTI13	DEBUG_BTN
PC14	I/O	RCC_OSC32_IN	Unusable
PC15	I/O	RCC_OSC32_OUT	Unusable
PF0	Output	GPIO_Output	RFID CS
PC0	Output	GPIO_Output	ROW4
PC1	Input	GPIO_Input	COL3
PA0	I/O	ADC1_IN0	Voltage
PA1	I/O	ADC1_IN1	Current
PA2	I/O	USART2_TX	UART_TX
PA3	I/O	USART2_RX	UART_RX
PA5	I/O	SPI1_SCK	RFID CK
PA6	I/O	SPI1_MISO	RFID MISO
PA7	I/O	SPI1_MOSI	RFID MOSI
PC4	Output	GPIO_Output	D4
PC5	I/O	GPIO_EXTI5	PUSHBTN
PB0	Input	GPIO_Input	COL2
PB13	I/O	SPI2_SCK	SD_SCK
PB14	I/O	SPI2_MISO	SD_MISO
PB15	I/O	SPI2_MOSI	SD MOSI
PA8	Output	GPIO_Output	D2
PA9	Input	GPIO_Input	COL1
PC6	Output	GPIO_Output	CS, SD
PC7	Output	GPIO_Output	ROW1
PA10	Output	GPIO_Output	D3
PA11	Output	GPIO_Output	ROW3
PA12	Output	GPIO_Output	ROW2
PA15	Output	GPIO_Output	RFID_RST
PB3	Output	GPIO_Output	D5
PB7	I/O	GPIO_EXTI7	RFID_IRQ
PB8	I/O	I2C1_SCL	OLED_SCL
PB9	I/O	I2C1_SDA	OLED_SDA

Amplitude Modulation and Phase Modulation Measurements

The amplitude and phase calculation data can be found at

<https://docs.google.com/spreadsheets/d/1cYFgEC03SYSkatNQ8QWacxegCii1hPyCMYheNmaAfrU/edit?usp=sharing>

Complete System Schematic



Power Supply

	8V, var	8V, DC	13V, var	13V, DC
5V	$\pm 22m, -16m$	5V	$\pm 20m, -9m$	5V
3.3V	$\pm 24m, -20m$	3.3V	$\pm 22m, -10m$	3.3V

Possibly? SD card drops by 60mV, for 200ms

V_I testing

AM & PM Done

Inst. values	meas	meas	meas	meas
$\frac{\pi}{2}$ Meas 1.575	0	0	0	0
$\frac{3\pi}{8}$ 1.170	207	206.8	10%	1516
$\frac{\pi}{4}$ 0.788	230	230.5	20%	3016
$\frac{\pi}{8}$ 0.395	253	254.1	30%	4503
0 ≈ 0.000 AVG			40%	6007
$-\frac{\pi}{8}$ -0.394			50%	7514
$-\frac{\pi}{4}$ -0.780			60%	9002
$-\frac{3\pi}{8}$ -1.178			70%	10493
$-\frac{\pi}{2}$ -1.575			80%	11997
			90%	13505
			100%	15045

meas	meas
$\frac{\pi}{2}$ 1.572	0
$\frac{3\pi}{8}$ 0.793	10
$\frac{\pi}{4}$ -0.02	20
$\frac{\pi}{8}$ -0.781	30
$-\frac{\pi}{2}$ -1.574	40
	50
	60
	70
	80
	90
	100

V₂ circuitry

(yellow V, blue I)

V clips ≈ 2.5 V_{pp}

I clips ≈ 2.2 V_{pp}

linearity ✓ (yellow top, blue bott, max range)

Steal last time's ADC V oscilloscope test

Load circuitry

Diode: peak forward current 1.56 V

settling: 2V

Inrush: 1.9V peak over 1k Ω cap

$$y = \frac{2\pi}{100} x - \pi$$

$$y' = \frac{\left(\frac{2\pi}{100} x - \pi\right) + \left(\frac{2\pi}{100} (x+1) - \pi\right)}{2}$$